

OE-EC704A — Web Technology

Semester VII • 3L:0T:0P • 3 Credits

Module 6 Exam Notes: String Handling

CGEC | B.Tech ECE (2023–27) | *Biraj Sarkar*

Contents

1	String Basics and Immutability	2
1.1	String Constant Pool (SCP)	2
2	String vs. StringBuffer	2
3	Constructors and Core Operations	3
3.1	String Constructors	3
3.2	Core String Operations	3
4	Utility Conversions	3
5	StringBuffer Operations	4

1. String Basics and Immutability

In Java, strings are represented by the `String` class inside the `java.lang` package.

String Immutability Concept

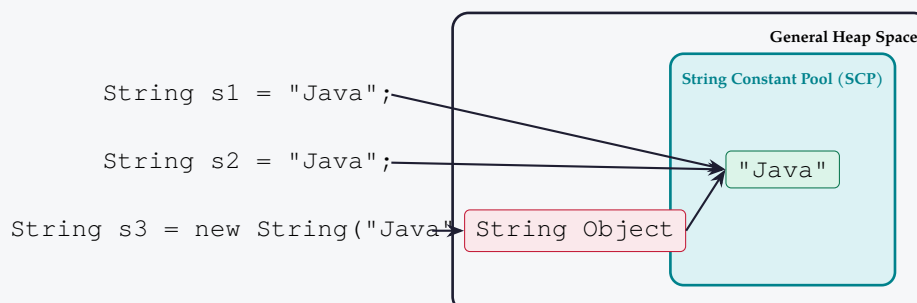
Java `String` objects are **immutable**, meaning their values cannot be changed once they are created in memory. Any modification operation (like concatenation or case conversion) does not alter the original string; instead, it returns a reference to a completely new `String` object.

1.1 String Constant Pool (SCP)

To save memory and increase performance, JVM maintains a special memory area inside the heap called the **String Constant Pool (SCP)**.

- **Literal Allocation:** When a string is created using a literal (e.g., `String s1 = "Java";`), JVM first checks the SCP. If the string is already present, it returns the existing reference. Otherwise, it allocates a new string in the pool.
- **New Object Allocation:** Using the `new` keyword (e.g., `String s2 = new String("Java");`) bypasses pool checking and creates a new object in the general heap space, which references the literal value in the SCP.

String Memory References: General Heap vs. String Constant Pool (SCP)



2. String vs. StringBuffer

When a program performs frequent modifications to character sequences, using the immutable `String` class generates high amounts of garbage references. Java provides `StringBuffer` to resolve this.

Metric	String Class	StringBuffer Class
Mutability	Immutable (read-only).	Mutable (can modify in-place).
Performance	Slower for modifications.	Fast and memory efficient for loops.
Thread Safety	Naturally thread-safe.	Thread-safe (methods are synchronized).
Memory Allocation	Allocations inside the SCP or Heap.	Allocated strictly in the general Heap.

3. Constructors and Core Operations

3.1 String Constructors

- `String()`: Initializes a newly created, empty String object.
- `String(char[] chars)`: Allocates a new String from an array of characters.
- `String(String original)`: Creates a copy of the argument string.

3.2 Core String Operations

Operation	Method Signature	Functional Description
Extraction	<code>char charAt(int index)</code> <code>int length()</code>	Returns the character at the specified index. Returns the character count.
Comparison	<code>boolean equals(Object obj)</code> <code>int compareTo(String another)</code>	Value-based comparison for character equality. Lexicographically compares two strings (returns difference).
Searching	<code>int indexOf(String str)</code> <code>boolean contains(CharSequence s)</code>	Returns the index of the first occurrence. Returns true if the string matches the pattern.
Modifying	<code>String substring(int start, int end)</code> <code>String trim()</code>	Returns characters from start (incl.) to end (excl.). Removes leading and trailing whitespace.

4. Utility Conversions

Java provides two major methods to perform conversions to strings:

- **toString() Method:** Defined in the root `Object` class. Every Java class inherits it and can override it to provide a custom string representation of the object's fields.

- **String.valueOf() Static Method:** Overloaded methods to convert primitive data types (like `int`, `double`, `boolean`) or objects into string representations. If an object is passed, it returns `obj.toString()` or `"null"` safely if the reference is null.

5. StringBuffer Operations

`StringBuffer` maintains a growable character buffer, defaulting to 16 characters of extra capacity.

- **append():** Appends data to the end of the buffer.
- **insert():** Inserts data at a specified index.
- **reverse():** Reverses the sequence of characters in-place.
- **Capacity vs. Length:** `length()` returns the actual number of characters stored, while `capacity()` returns the total allocated buffer space.

Quick Revision — Module 6

String Handling | OE-EC704A

String & StringBuffer Syntax

Operation	Syntax and Example Usage
Literal Assignment	<code>String s1 = "Biraj";</code> // Allocated in SCP
Object Constructor	<code>String s2 = new String("Biraj");</code> // Heap object reference
Character Extraction	<code>char c = s1.charAt(2);</code> // 'r'
Substring Extraction	<code>String sub = s1.substring(1, 3);</code> // "ir"
StringBuffer Operations	Mutable buffer syntax
Declaration	<code>StringBuffer sb = new StringBuffer("CGE");</code>
Append Operation	<code>sb.append("C");</code> // sb becomes "CGEC"
Insert Operation	<code>sb.insert(3, "-");</code> // sb becomes "CGE-C"
In-place Reverse	<code>sb.reverse();</code> // sb becomes "C-EGC"

One-Line Definitions

- **Immutability:** State constraint preventing modification of heap values after initial allocation.
- **String Constant Pool:** A cached region in Java heap memory storing unique string literals.
- **StringBuffer:** A thread-safe, synchronized, mutable character sequence class.
- **StringBuilder:** A non-synchronized, fast, mutable character sequence class.
- **equals():** Overridden method in String checking value-based character equality.
- **== Operator:** Operator checking whether two reference variables point to the same memory address.
- **valueOf():** Static String utility method converting primitives and objects to strings.
- **toString():** Standard Object method returning class representation details as a String.

Frequently Asked / Viva Questions

Q1. Why are String objects immutable in Java?

To enable String Constant Pool caching (saving heap memory), ensure thread-safety, and prevent security vulnerabilities (since connection strings and file paths cannot be altered).

Q2. What is the difference between `equals()` and `==` when comparing Strings?

`==` compares reference equality (whether variables point to the exact same object)

Frequently Asked / Viva Questions (contd.)

reference). The `equals()` method checks value equality (whether character sequences are identical).

Q3. How many objects are created in the statement: `String s = new String("Hello");`?

Two objects. One literal object is placed in the String Constant Pool (if not already present), and a second String object is created in the general heap space.

Q4. What does the method `intern()` do?

It returns the canonical representation of the string from the String Constant Pool. If the pool already contains the string, it returns its reference; otherwise, it adds the string to the pool and returns the reference.

Q5. What is the difference between `StringBuffer` and `StringBuilder`?

Both are mutable classes. `StringBuffer` is thread-safe and has synchronized methods, causing high performance overhead. `StringBuilder` is not synchronized, making it faster and preferred for single-threaded applications.

Q6. What is the default capacity of a `StringBuffer`? How does it increase?

The default initial capacity is 16. When the buffer capacity is exceeded, it grows automatically using the formula: $\text{New Capacity} = (\text{Old Capacity} \times 2) + 2$.

Q7. Can we override the static method `valueOf()` in custom classes?

No. Static methods belong to the class scope and are resolved statically during compilation. They cannot be overridden in subclasses. Custom classes override `toString()` instead.

Q8. What does the method `compareTo()` return?

It returns 0 if strings are equal, a negative integer if the current string is lexicographically smaller than the argument, and a positive integer if it is larger.

Q9. Why does `System.out.println(obj)` invoke the `toString()` method?

The print streams implicitly check if the argument is an object. If yes, it calls `String.valueOf(obj)`, which internally invokes `obj.toString()` to get a printable string.

Q10. What is the purpose of the `trim()` method in the `String` class?

It returns a copy of the string with all leading and trailing whitespaces (characters with Unicode code point less than or equal to 'U+0020') removed.

Comparison: String vs. StringBuffer vs. StringBuilder

Metric	String	StringBuffer	StringBuilder
Mutability	Immutable.	Mutable.	Mutable.
Thread Safety	Yes (due to immutability).	Yes (synchronized methods).	No (non-synchronized).
Performance	Slowest for edits.	Slower (lock overhead).	Fastest.
Introduced in	Java 1.0	Java 1.0	Java 1.5